Requirements

---

1. Business Requirements

OrderFlow is a small shop that sells online. More customers are coming. The old system is a basic app that needs a server. The shop wants a new system on AWS that:

1. Allow an order to be created.
2. Assign a unique order identifier.
3. Store order information persistently.
4. Allow authorized users to retrieve an order.
5. Allow authorized users to retrieve multiple orders.
6. Allow order status to be updated.
7. Prevent invalid order-state transitions.
8. Allow appropriate orders to be cancelled.
9. Validate incoming requests.
10. Handle errors gracefully.
11. Automatically scale as request volume increases.
12. Avoid continuously running application servers.
13. Protect order data from unauthorized access.
14. Provide logging and monitoring.
15. Keep the architecture simple and cost-conscious.
16. Support infrastructure management through Terraform.
17. Be suitable for implementation as a small AWS project.

2. Technical Requirements

2.1 Tools & AWS-Services

- **API Gateway:** the front door
- **Lambda:** small programs that run only when needed
- **DynamoDB:** the database
- **IAM:** who may do what
- **CloudWatch:** logs and alarms
- **Terraform:** builds everything from text files
- **Python:** my code language for the Lambda functions

2.2 Technical rules

1. **API:** The system uses a REST API through Amazon API Gateway.
2. **Code:** AWS Lambda functions do the work. There are functions for create, read/list and status update.
3. **Database:** Orders are saved in DynamoDB. The system uses Query and does not use Scan for customer searches.
4. **Access control:** IAM protects the API. Each Lambda gets only the rights it needs (least privilege).
5. **Input check:** The system checks all incoming data. It sends clear HTTP codes (200, 201, 400, 404, 409, 500).
6. **Order rules:** The system allows only valid status changes. It uses conditional updates, so two changes at the same time cannot break an order.
7. **Monitoring:** The system writes logs to CloudWatch. It has basic alarms for errors.
8. **Infrastructure:** Terraform builds all core parts from code.
9. **Design:** The system is serverless and pay-per-request. No server runs all day.
10. **Language:** The Lambda code is written in Python.

3. Functional Requirements

1. Create an order
2. Give each order a unique ID
3. Save the order
4. Get one order
5. Get a list of orders
6. Change the order status
7. Refuse wrong status changes
8. Cancel an order (only early)
9. Check that the sent data is correct
10. Show clear error messages

3.1 Create Order

- **Endpoint:** POST /orders
- **Description:** The system provides an API endpoint to accept and process new order creations.
- **Request Payload:**

JSON

```
{
  "customerId": "C1001",
  "items": [
    {
      "productId": "P100",
      "quantity": 2
    }
  ],
  "totalAmount": 49.98
}
```

- **System Operations:**
    - Validate incoming payload structure, mandatory fields, and data types.
    - Generate a unique order identifier (e.g., UUID).
    - Assign the default initial status: PENDING.
    - Store order information persistently along with creation and update timestamps.
    - Return the complete created order details in the response payload.

3.2 Retrieve Order

- **Endpoint:** GET /orders/{orderId}
- **Description:** The system allows an authorized user to fetch a specific order by its unique ID.
- **Response Payload Content:**
    - Order ID
    - Customer ID
    - Items (productId, quantity)
    - Total Amount
    - Status
    - Creation Time (createdAt)
    - Last Updated Time (updatedAt)

3.3 Retrieve Orders

- **Endpoint:** GET /orders
- **Description:** The system provides a scalable mechanism to query and retrieve multiple orders.
- **Database & Scalability Considerations:**
    - Efficient access paths must be supported using index-based queries (e.g., querying by customerId using a Global Secondary Index).
    - Supports pagination (e.g., limit and nextToken / offset) to avoid full table scans (SCAN) and protect system scalability.

3.4 Update Order Status

- **Endpoint:** PATCH /orders/{orderId}/status
- **Description:** Allows authorized users or administrative services to update the state of an existing order.
- **Request Payload:**
- JSON

```
JSON

{
  "status": "CONFIRMED"
}
```

- **Supported Order States:**
  - PENDING
  - CONFIRMED
  - PROCESSING
  - SHIPPED
  - DELIVERED
  - CANCELLED

3.5 Order-State Validation & Business Rules

- **Description:** The system enforces strict business logic to validate status transitions prior to mutating database records.
- **Allowed State Transitions:**
  - PENDING → CONFIRMED
  - PENDING → CANCELLED
  - CONFIRMED → PROCESSING
  - CONFIRMED → CANCELLED
  - PROCESSING → SHIPPED
  - SHIPPED → DELIVERED
- **Business Logic Enforcement:**
  - Invalid state transitions (e.g., DELIVERED → CANCELLED or CANCELLED → SHIPPED) must be rejected at the application level.
  - The system must return an explicit HTTP response (e.g., 400 Bad Request or 422 Unprocessable Entity) indicating that the requested state transition is invalid, leaving the persisted order state unchanged.

4. Non-Functional Requirements

4.1 Scalability

- **Requirement:** Handle growing API traffic seamlessly without manual server provisioning or application server management.
- **Project Implementation:** Uses **Amazon API Gateway**, **three dedicated AWS Lambda functions**, and **Amazon DynamoDB with on-demand billing** (`PAY_PER_REQUEST`). The entire stack scales automatically in response to incoming request spikes without requiring server management.

4.2 Availability

- **Requirement:** Ensure high system uptime through managed, resilient infrastructure.
- **Project Implementation:** Relies on **Amazon API Gateway**, **AWS Lambda**, and **Amazon DynamoDB**, which are fully managed AWS services distributed across multiple Availability Zones. *(Note: This high-availability model applies to the AWS cloud deployment; local execution via LocalStack is for development only).*

4.3 Security

- **Requirement:** Prevent direct database access and safeguard order data across all access layers.
- **Project Implementation:**
    - **API Client Layer:** Clients connect exclusively through **API Gateway** over **HTTPS** using **SigV4 / IAM authorization**.
    - **Database Layer: Lambda execution roles** enforce least-privilege IAM policies to access DynamoDB, ensuring clients never touch the database directly.
    - **Data at Rest: DynamoDB encryption at rest** is enabled by default.

4.4 Performance

- **Requirement:** Deliver fast, low-latency execution for high-frequency operations like retrieving order details.
- **Project Implementation:**
    - **Single Order Fetch (`GET /orders/{orderId}`):** Performs a direct, single-digit millisecond DynamoDB `GetItem` lookup using the primary key.
    - **Multiple Orders Fetch (`GET /orders`):** Queries a dedicated **Global Secondary Index (`CustomerCreatedIndex`)** instead of performing an expensive, unindexed table scan.

4.5 Reliability & Data Consistency

- **Requirement:** Prevent concurrent requests or stale updates from accidentally overwriting newer order states.
- **Project Implementation:** Status update operations enforce optimistic concurrency control by checking both the **expected current status** and a **version attribute**. DynamoDB applies updates conditionally only if both parameters match, preventing race conditions.

4.6 Cost Optimization

- **Requirement:** Eliminate idle costs incurred by continuously running application infrastructure.
- **Project Implementation: AWS Lambda** executes strictly on demand, and **DynamoDB** uses **pay-per-request billing**. Zero continuously running application servers or provisioned database capacity exist, minimizing cost during idle periods.

4.7 Maintainability & Architecture

- **Requirement:** Organize source code cleanly to isolate routing, business logic, and database operations.
- **Project Implementation:**
    - **API Route Handlers:** API Gateway routes traffic to separate, focused Lambda handlers for create, read/list, and update workflows.

- **Shared Domain Logic:** Shared request validation, order state business rules, and database operation helpers are centralized within `src/order_api/common.py`.